INSTRUCTOR LIVE SCRIPT

WEEK 1 · WEDNESDAY

MODULE 00 CLINIC + CURSOR COUNTDOWN DEMO

Follow this page-by-page while projecting. Students copy you. Core path = labs 00:00–04.

WED · 6:00–8:00 PM · ZOOM · CLINIC, NOT LECTURE

Tone: Lab clinic. Screenshots stay local if D2L Dropbox isn't ready. Define acronyms once (IDE, WSL, SSH). Mac = **Cmd** wherever Windows uses **Ctrl**.

6:00 — OPEN + CHECK LAB 00:00 · ~15 MIN

> **SAY —** Tonight is a lab clinic. We're not covering new theory — we're getting VS Code working on every laptop. Labs gate progress, not attendance. Sign in now: tktechventure.com/agentic-coding/sign-in.

DO

- Attendance sign-in · OS check in chat (Windows / Mac / Ubuntu)
- Catch-up: no VS Code → `code.visualstudio.com` → install → **Skip sign-in**

> **SAY —** An IDE is a text editor plus a terminal plus tools in one window. Not Google Docs — files live on your disk, and the terminal talks to your computer.

DONE LOOKS LIKE

Welcome tab + full-window screenshot.

6:15 — UI TOUR LAB 00:01 · ~20 MIN

> **SAY —** Five regions — say them with me: Activity Bar, Side Bar, Editor, Panel/Terminal, Status Bar.

DO LIVE (STUDENTS COPY)

- **File → Open Folder** → create `agentic-coding` (or any empty folder)
- Command Palette: Windows `Ctrl+Shift+P` · Mac `Cmd+Shift+P`
- Settings → Font Size → **16**
- Pause: "Activity bar = icons on the left. Side bar = the panel that opens."

DONE LOOKS LIKE

Folder open + font size 16 in Settings.

6:35 — TERMINAL LAB 00:02 · ~25 MIN

> **SAY —** Editor = where you write files. Terminal = where you run commands. If you type in the wrong place, nothing happens — click the bottom panel.

DO

- Open terminal: `Ctrl+\`` / `Cmd+\`` or **View → Terminal**
- Run twice: `echo hello`
- Screenshot: both commands + output visible

> **SAY —** We are not learning cd and ls tonight — that's Module 01. Tonight: prove the terminal works.

7:00 — EXTENSIONS LABS 00:03–04 · ~25 MIN

> **SAY —** Out of the box, VS Code doesn't know Python. Extensions add powers. Install only what I say — random extensions break setups.

DO

- Extensions (`Ctrl+Shift+X` / `Cmd+Shift+X`)
- Install **Python** — publisher **Microsoft** only
- Install **Docker** — publisher **Microsoft**
- Open the whale sidebar

> **SAY —** Empty Docker sidebar or "Failed to connect" is **correct**. Docker Engine is Module 02. Tonight we only install the extension.

7:25 — WSL CLINIC LAB 00:05 · ~25 MIN

WINDOWS

- Install **WSL** extension (Microsoft)
- Green bottom-left → **WSL: Ubuntu**
- Terminal: `uname -a` (should say Linux)

MAC / LINUX

You're already on a real shell. Run `uname -a`. Buddy-help a Windows neighbor.

STUCK-SPOT CHEAT SHEET

WHAT FAILS	FIX LINE
Sign-in / sync	Skip — optional. Never required for Phase 1.
Mac hitting Ctrl	Mac = Cmd tonight (`Cmd+Shift+P`).
Typing in editor	Click the bottom panel; or View → Terminal.
Docker "Failed to connect"	Correct — Engine is Module 02.
WSL password blank	Invisible typing — keep typing; reboot after install if needed.
10 random extensions	Only Python + Docker (+ WSL / Remote SSH). Continue.dev comes later.

7:50 — CLOSE ~10 MIN

> **SAY —** Done for tonight = VS Code opens, terminal echoes, Python + Docker extensions installed, Windows on green WSL if you got that far. Through Sunday: finish remaining Module 00. Don't stress Docker Engine or full terminal yet. Week 2 = remote connections + Module 00 gate.

TIMING TIP

If the clinic is behind, skip the Cursor demo on the next page and only show a pre-built countdown once. Core path stays **00:00–04**; Cursor / venv is gravy.

OPTIONAL STRETCH DEMO

CURSOR · COUNTDOWN + VENV + PIP

Use after core labs, or as a 10–15 min preview. Say clearly: full venv labs land later (Module 03).

WHY BLOCK — SAY BEFORE TYPING

Why a venv?

System Python is shared — global installs let Project A break Project B. A venv is a private toolbox for **this** project.

Why requirements.txt + pip?

`requirements.txt` = shopping list. `pip install -r …` installs into **this** venv so any machine can rebuild it.

Why Cursor here?

Same folder/terminal habits as VS Code. Tonight: folder → venv → install → run. AI help comes later.

DEMO PROJECT — BUILD LIVE

1. FOLDER (PREFER WSL UBUNTU ON WINDOWS)

```
mkdir -p ~/countdown-demo && cd ~/countdown-demo
```

REQUIREMENTS.TXT

```
colorama==0.4.6
```

COUNTDOWN.PY

```
import time
from colorama import Fore, Style, init
init(autoreset=True)

def countdown(seconds: int) -> None:
    print(Fore.CYAN + f"Countdown from {seconds}...")
    for i in range(seconds, 0, -1):
        print(Fore.YELLOW + str(i)); time.sleep(1)
    print(Fore.GREEN + "Go!")

if __name__ == "__main__":
    countdown(5)
```

2-4. CREATE VENV · INSTALL · RUN

```
python3 -m venv .venv
source .venv/bin/activate          # prompt shows (.venv)
pip install -r requirements.txt
python countdown.py                # 5…4…3…2…1… Go!
```

SAY — colorama is not built into Python. Without venv + pip, *`import colorama`* fails — that's the "why."

5. OPTIONAL 60-SECOND PROOF

```
deactivate && python countdown.py   # often ImportError
source .venv/bin/activate && python countdown.py   # works
```

SAY — Same file. Different environment. That's why we care about venv before real apps.

ZOOM / CHAT CHEAT SHEET

```
cd ~/countdown-demo && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt && python countdown.py
```

Windows PowerShell activate: `.venv\Scripts\Activate.ps1`